10 | 基础组件（上）：后端业务基础设施

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

工程初始化完成了，Hify 前后端都能跑，`make start` 一键启动。但现在还不能直接写业务，“空项目”还缺一层东西：**基础组件**。

08 讲搭 hify-common 的时候，我们做了配置层面的基础，MyBatis-Plus 的分页插件和自动填充、Redis 的 RedisTemplate 和序列化配置。但业务封装层还没做，比如：

1. BaseEntity 基类怎么定
2. 分页查询怎么从前端参数转成 PageResult
3. 接口入参怎么校验
4. 调外部 LLM API 的 HTTP 客户端怎么封装
5. 线程池怎么隔离

6. 熔断怎么配

这些是所有业务模块都会用到的底层能力，不先搭好，后面每个模块都要重复处理一遍。但问题来了：**具体需要准备哪些基础组件？你可能有经验能列出一部分，但难免有遗漏**。这一讲引入一个新的协作模式来解决这个问题。

新的协作模式：先问再做

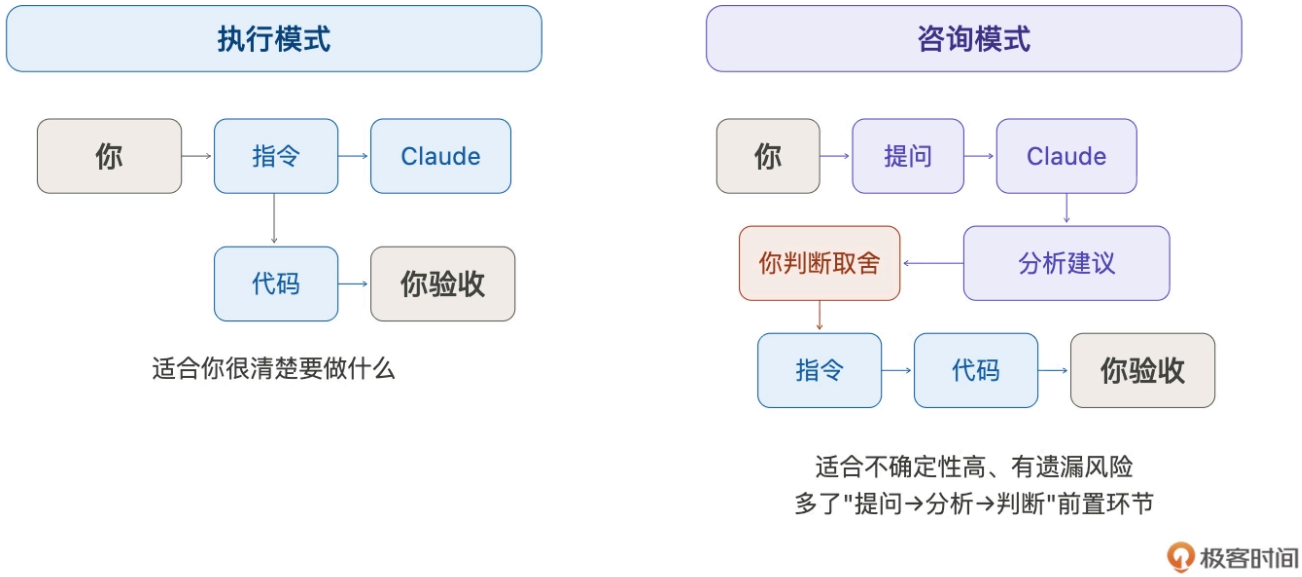
回顾前面几讲，我们和 Claude Code 的协作基本都是一个模式：**你想清楚了，让它做**。“按照规范实现这个接口”，“帮我生成 Maven 骨架”……你知道要做什么，它负责执行。

但现在的情况不一样，“业务开发前需要准备哪些基础组件”这个问题，你有大致想法，但不确定有没有遗漏、先后顺序对不对。

这时候换一种模式：**先让 Claude Code 帮你想，再让它做**。不是让它替你做决策，而是**把它当成一个资深架构师来咨询**，你问“这一步应该考虑什么”，它帮你梳理全景，你判断哪些要做、哪些不做，然后再进入执行。

我把这两种模式叫**执行模式**和**咨询模式**：

> 执行模式：你想清楚了 → 给指令 → Claude Code 做 → 你验收。适合你对任务很清楚的时候。
>
> 咨询模式：你知道大方向但不确定细节 → 先问 Claude Code → 它帮你梳理 → 你判断取舍 → 再给指令执行。适合不确定性高的时候。



这个方法论后面会反复用到。每次进入新阶段（新模块、测试、部署），都可以先用咨询模式问一句“这个阶段我应该考虑什么”，确保不遗漏。

让 Claude Code 梳理基础组件清单

直接用。我问 Claude Code：

> Hify 项目工程骨架已经搭好（Maven 多模块、hify-common 的 Result / 异常处理 / MyBatis-Plus 配置 / Redis 配置、前端 Vue 工程）。现在要开始做业务功能了。在写业务代码之前，还需要准备哪些基础组件？从数据库层、接口层、外部调用、缓存、可观测性几个角度帮我梳理，每个组件说明它解决什么问题。

它给了一份很详细的清单。我就不放细节了，就放一个总结：



有些是我预期到的（线程池、熔断），但有几个是我自己列清单时会漏掉的。我按优先级做了分档，不是所有东西都要现在做，有些不做业务就跑不起来，有些可以写业务时同步补。

第一档：必须先做，否则业务代码跑不起来

1. 数据库初始化脚本（schema.sql）。这是我差点忽略的，现在还没有建表 DDL。Spring Boot 启动时需要库和表已存在，否则 Mapper 跑不起来。需要创建所有业务表的 DDL 文件。
2. @MapperScan 扫描路径。目前启动类的扫描路径只覆盖了 `com.hify.app`，但业务 Mapper 在各子模块里（`com.hify.provider.mapper` 等），Bean 注册不上。不改这个，所有 Mapper 注入都会报错。

3. llmExecutor / asyncExecutor 线程池。CLAUDE.md 要求 LLM 调用必须用 `@Qualifier("llmExecutor")`，Chat 模块注入时这个 Bean 必须存在。

第二档：做业务时同步补，第一个列表接口之前搞定

1. BaseEntity（公共字段封装）。每个实体都有 id、created_at、updated_at、deleted，不封装就每个实体重复写。
2. 统一分页参数封装。几乎每个列表接口都要分页，把前端参数转成 MyBatis-Plus 的 Page 对象，封装一次全部复用。
3. 入参校验（@Valid）。每个创建 / 更新接口都需要参数校验，和 08 讲的全局异常处理器串起来。
4. 统一时间序列化。不配的话 Jackson 序列化 LocalDateTime 默认输出数组格式 `[2025,3,16,10,30,0]`，前端解析不了。
5. Spring Cache 集成。CLAUDE.md 规定 Provider / Agent 配置走 Redis Cache-Aside，TTL 30min。现在只有裸 RedisUtil，没有启用 `@EnableCaching` + `RedisCacheManager`，`@Cacheable` 注解用不了。

第三档：可以后补，不影响功能开发

1. HTTP 客户端封装。调 LLM API 的基础，但 Chat 模块开发时再做也来得及。
2. Resilience4j 熔断配置。04 讲定了每个提供商独立熔断器，但 Chat 模块完成后补也不迟。
3. 结构化日志配置。logback-spring.xml 区分开发 / 生产环境，请求日志过滤器记录 method、path、耗时。联调阶段前补上。

Claude Code 帮我发现了两个我自己会漏掉的：**schema.sql 和 @MapperScan**。不是什么高深的东西，但你在列基础组件清单时很容易忽略。脑子里想的都是 BaseEntity、线程池这些有设计感的东西，反而忘了最基础的东西，比如表还没建、扫描路径不对。等到启动报错才发现，浪费的是排查时间。

这就是咨询模式的核心价值，用它做遗漏检查。Claude Code 见过的项目比你多，它的建议不能直接照搬，但用来查漏补缺非常好。

我的决定是：这一讲把三档全做了。 虽然第三档可以后补，但提前搭好，后面就不用回头。按依赖关系排序：先做第一档（跑起来），再做第二档（业务能写），最后补第三档（健壮性），DemoItem 验收放在第二档做完之后。

第一档：让业务代码能跑起来

1. 数据库初始化脚本

Claude Code 提示词是：

> 按照 CLAUDE.md 的数据库规范和数据模型，生成所有业务表的建表 DDL。放在 hify-app/src/main/resources/db/schema.sql。表名小写下划线、主键 id bigint 自增、时间字段 created_at/updated_at datetime、逻辑删除 deleted tinyint 默认 0、字符集 utf8mb4。包含：provider、model_config、agent、agent_tool、mcp_server、chat_session、chat_message。

效果是：

schema.sql 已生成，7 张表：

表	说明	关键索引
`provider`	模型提供商	`UNIQUE(name)`
`model_config`	模型配置，1:N 挂在 provider 下	`KEY(provider_id)`
`mcp_server`	MCP 工具服务	`UNIQUE(name)`
`agent`	Agent 配置，绑定 model_config	`UNIQUE(name)` 、 `KEY(model_config_id)`
`agent_tool`	Agent ↔ MCP Server M:N 关联	`UNIQUE(agent_id, mcp_server_id)`
`chat_session`	对话会话，挂在 agent 下	`KEY(agent_id)`
`chat_message`	对话消息，增长最快	`KEY(session_id)`

几点说明：

1. **`api_key` 放在 `model_config`** — 不同模型配置可能用不同 key，比 provider 粒度更合适；provider 表只存连接信息
2. **`chat_message` 无 `deleted` 以外的大字段索引** — content 是 `LONGTEXT`，查询走 `session_id` 游标分页，避免 `COUNT(*)`（符合 CLAUDE.md 规范）
3. **`agent_tool` 无逻辑删除** — 关联关系直接物理删除即可，不需要软删

这个就不展开说明了，大家可以自己琢磨一下细节。

2. @MapperScan 扫描路径

这个改动就一行代码，但不改所有 Mapper 都注入不了：

> 在 HifyApplication 启动类上加 @MapperScan(“com.hify.**.mapper”)，扫描所有子模块的 Mapper 包。

效果是：

```
package com.hify.app;    "hify": Unknown word.

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication(scanBasePackages = "com.hify")    "hify": Unknown word.
@MapperScan("com.hify.**.mapper")    "hify": Unknown word.
public class HifyApplication {    "Hify": Unknown word.

    public static void main(String[] args) {
        SpringApplication.run(HifyApplication.class, args);    "Hify": Unknown word.
    }
}
```

3. 线程池配置

场景：用户在对话（调 LLM，卡了 30 秒），同时另一个用户打开管理页面看 Agent 配置。如果共用线程池，对话请求占满了所有线程，管理页面一直转圈。

> 在 hify-common 中创建 ThreadPoolConfig（com.hify.common.config）。定义两个线程池：llmExecutor（核心 10，最大 50，队列 100，线程名前缀 llm-，拒绝策略 CallerRunsPolicy）用于 LLM 调用；asyncExecutor（核心 5，最大 20，队列 200，线程名前缀 async-）用于日志异步写入等非关键任务。用 @Bean + @Qualifier 注册。

参数依据：目标 20-50 人同时在线，一半的人同时对话就是 10-25 个并发 LLM 调用，核心线程 10 够用，最大 50 留余量。一期先这样，后面根据监控调整。

线程名前缀很重要，后面看日志时，`llm-3` 立刻知道是 LLM 调用线程，`async-1` 是异步任务线程。不设前缀全是 `pool-1-thread-3`，排查时分不清。

复制代码

```java
package com.hify.common.config;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.concurrent.Executor;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

@Configuration
public class ThreadPoolConfig {

    @Bean
    @Qualifier("llmExecutor")
    public Executor llmExecutor() {
        return new ThreadPoolExecutor(
                10, 50,
                60L, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>(100),
                new NamedThreadFactory("llm-"),
                new ThreadPoolExecutor.CallerRunsPolicy()
        );
    }

    @Bean
    @Qualifier("asyncExecutor")
    public Executor asyncExecutor() {
        return new ThreadPoolExecutor(
                5, 20,
                60L, TimeUnit.SECONDS,
                new LinkedBlockingQueue<>(200),
                new NamedThreadFactory("async-"),
                new ThreadPoolExecutor.AbortPolicy()
        );
    }
}
```

到这里，`mvn spring-boot:run` 应该能正常启动、Mapper 能注入、线程池 Bean 就绪。跑一下确认没有启动报错，再进入第二档。

第二档：业务开发的基础能力

接下来，我就不展示每个指令的执行效果了，比较复杂的我会展开说明。

1. BaseEntity

每个数据库实体都有公共字段。不封装的话，每个 Entity 都要重复写 id、created_at、updated_at、deleted 和对应的注解。

> 在 hify-common 中创建 BaseEntity 类。字段：id（Long，@TableId 自增）、createdAt（LocalDateTime，插入时自动填充）、updatedAt（LocalDateTime，插入和更新时自动填充）、deleted（Integer，@TableLogic，默认 0）。后面所有业务实体继承这个类。

2. 统一分页封装

08 讲定义了分页的请求参数（page、pageSize）和响应格式（PageResult），也配好了分页插件，但还没有把它们串起来。

> 在 hify-common 中创建 PageHelper 工具类。提供两个静态方法：
> toPage(page, pageSize) 把前端参数转成 MyBatis-Plus 的 Page 对象（page 从 1 开始，pageSize 默认 20 最大 100）；toPageResult(IPage) 把查询结果转成我们的 PageResult。

3. 入参校验

每个创建和更新接口都需要校验入参。Spring Boot 的 `@Valid` + JSR 303 注解就够用，关键是和 08 讲的 GlobalExceptionHandler 配合好。它已经捕获了 MethodArgumentNotValidException，会转成 `Result.fail(ErrorCode.PARAM_ERROR, 具体校验信息)`。

4. 统一时间序列化

> 在 hify-common 中配置 Jackson 的全局时间序列化。LocalDateTime 统一用 ISO 8601 格式（yyyy-MM-dd'T'HH:mm:ss），LocalDate 用 yyyy-MM-dd。配置

> JavaTimeModule，关掉 WRITE_DATES_AS_TIMESTAMPS。

一个配置类，十几行代码，但省掉后面无数前端时间解析的坑。

5. Spring Cache 集成

08 讲配好了 RedisTemplate 和 RedisUtil，但只是裸的 get/set 操作。业务层需要 `@Cacheable`、`@CacheEvict` 这些声明式缓存。Provider 配置读多写少，加个注解就自动缓存，不需要手动写 if-else。

> 在 hify-common 中启用 Spring Cache。@EnableCaching，配置 RedisCacheManager，默认 TTL 30 分钟，key 前缀 hify:。配置不同缓存名的 TTL：provider-cache 30 分钟、agent-cache 30 分钟、session-cache 2 小时。

这样后面业务 Service 直接加 `@Cacheable(cacheNames = "provider-cache")` 就行，缓存的读写、过期、序列化全自动处理。写入时加 `@CacheEvict` 失效缓存，标准的 Cache-Aside 模式。

6. CRUD 标准流程验证

第二档的所有组件搭好了，用一个 DemoItem 跑通完整 CRUD，验证它们协同工作。

> 实现一个最简单的 CRUD 演示。实体：DemoItem，只有 name（String）和 status（Integer）两个字段，继承 BaseEntity。完整实现 Controller → Service → ServiceImpl → Mapper → Entity → CreateReq/UpdateReq/Resp DTO。Controller 使用 RESTful 路径 /api/v1/demo-items，返回统一 Result，列表接口返回 PageResult，创建和更新接口使用 @Valid 校验，Service 层处理业务逻辑，Mapper 继承 BaseMapper。

验收：

复制代码

```
# 入参校验生效吗?
```

```
curl -X POST http://localhost:8080/api/v1/demo-items \
  -H "Content-Type: application/json" \
  -d '{"name": "", "status": 1}'
# 期望：Result.fail，提示 name 不能为空

# 正常创建
curl -X POST http://localhost:8080/api/v1/demo-items \
  -H "Content-Type: application/json" \
  -d '{"name": "测试项", "status": 1}'
# 期望：Result.ok(id)，数据库 created_at 自动填充

# 分页列表
curl "http://localhost:8080/api/v1/demo-items?page=1&pageSize=10"
# 期望：PageResult，时间字段是 ISO 8601 格式

# 逻辑删除
curl -X DELETE http://localhost:8080/api/v1/demo-items/1
# 期望：数据库 deleted=1，再查列表看不到了
```

每一个请求验证一个基础组件：

空 name → 入参校验 + 全局异常处理器

创建成功 → BaseEntity 自动填充

列表 → PageHelper + 分页插件

时间格式 → Jackson 配置

逻辑删除 → MyBatis-Plus 配置

全部通过，说明第二档组件就绪。DemoItem 留着不删，后面做 Provider 模块时，它是最好的参考模板。

第三档：健壮性补齐

这些不做也能写业务，但从架构和实现的角度，最好提前搭好，后面不用回头。这也是培养咱们设计和实现系统的好习惯。

1. HTTP 客户端封装

Hify 需要调用各种外部 LLM API。先用咨询模式确认方案：

> 帮我分析普通请求用 RestTemplate、流式 SSE 请求用 OkHttp EventSource 这个方案，有没有问题?

Claude Code 确认方案合理，提醒了连接池要分开配、超时参数要区分。进入执行模式：

> 在 hify-common 中创建 LlmHttpClient 类（com.hify.common.http）。内部持有 RestTemplate（连接超时 5s，读超时 60s）和 OkHttpClient（连接超时 5s，读超时 120s）。提供 post(url, headers, body) 方法返回 String，提供 stream(url, headers, body, callback) 方法通过回调逐行返回。所有请求记录日志（URL、耗时、状态码），异常统一转为 LlmApiException（区分 TIMEOUT、AUTH_FAILED、RATE_LIMITED）。

输出是：

```java
/**
 * 同步 POST，返回响应体字符串。
 * 连接超时 5s，读超时 120s。
 */
public String post(String url, Map<String, String> headers, String body) {
    Request request = buildRequest(url, headers, body);
    long start = System.currentTimeMillis();

    try (Response response = httpClient.newCall(request).execute()) {
        long elapsed = System.currentTimeMillis() - start;
        log.info("LLM POST {} status={} cost={}ms", url, response.code(), elapsed);

        checkStatus(response.code(), url);

        ResponseBody responseBody = response.body();
        return responseBody != null ? responseBody.string() : "";
    } catch (SocketTimeoutException e) {
        log.warn("LLM POST {} timeout after {}ms", url, System.currentTimeMillis() - start);
        throw new LlmApiException(LlmApiException.Type.TIMEOUT, "LLM 请求超时: " + url, e);
    } catch (LlmApiException e) {
        throw e;
    } catch (IOException e) {
        log.error("LLM POST {} error: {}", url, e.getMessage());
        throw new LlmApiException(LlmApiException.Type.UNKNOWN, "LLM 请求异常: " + e.getMessage(), e);
    }
}
```

上面是生成的局部代码。是不是很完整，比我写的都好。

这里有个点，为什么超时设这么长？**LLM API 不是普通 HTTP 调用，GPT-4 一次响应可能 10–20 秒，流式可能持续一两分钟。60 秒和 120 秒是实际经验的平衡**。这也是我们作为程序员的价值所在，我们会去理解需求，从宏观去思考超时时间是多久。因为 AI 不知道 GPT-4 一次响应可能 10–20 秒，这是物理世界的表现。

2. Resilience4j 熔断与重试

线程池解决了“一个 LLM 卡了不影响其他功能”的问题。熔断解决另一个问题：某个提供商整个挂了，系统还在不停发请求，每次等 60 秒超时——挂了就是挂了，等也白等。

> 在 hify-common 中配置 Resilience4j 熔断器（com.hify.common.resilience）。application.yml 配置：slidingWindowSize 10，failureRateThreshold 50%，waitDurationInOpenState 30s，permittedNumberOfCallsInHalfOpenState 3。创建 CircuitBreakerService，按 providerName 获取或创建独立熔断器实例。重试逻辑：网络超时重试 2 次（间隔 1s），限流退避重试（2s、4s），认证失败不重试。

输出是：

编译通过。整体设计：

application.yml — Resilience4j 全局配置，定义了 `llm-default` 熔断配置模板和两套重试配置（`timeout-retry` / `rate-limit-retry`）。

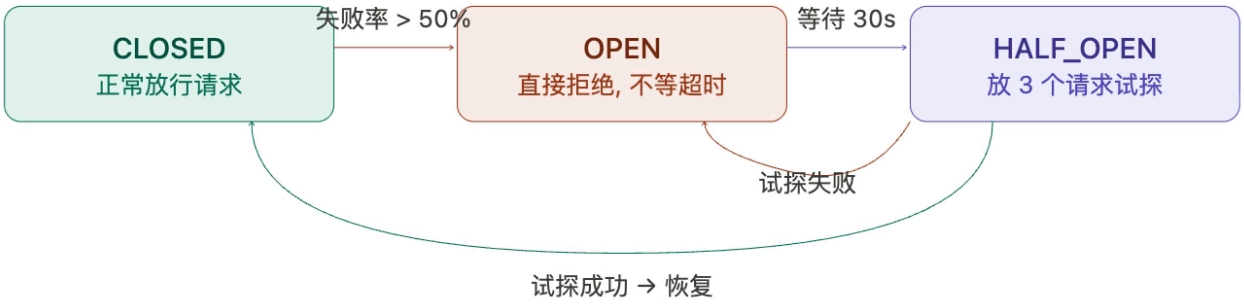
CircuitBreakerService.java — 核心逻辑：

关注点	实现
每个 Provider 独立熔断器	`ConcurrentHashMap` 按 `providerName` 懒创建
熔断触发条件	10 次滑动窗口，失败率 ≥50%，开路 30s，半开测试 3 次
AUTH_FAILED 不计入失败率	`recordException` 谓词过滤
超时重试	固定间隔 1s，最多重试 2 次（共 3 次尝试）
限流退避重试	指数退避 2s → 4s，最多重试 2 次
认证失败不重试	两个 `retryOnException` 谓词均不匹配 `AUTH_FAILED`，直接穿透抛出
调用顺序	`CircuitBreaker` 包 `TimeoutRetry` 包 `RateLimitRetry` 包业务调用

使用方式：

```
String result = circuitBreakerService.execute("openai", () ->
    llmHttpClient.post(url, headers, body)
);
```

下图是熔断器状态流转图，你可以看看，理解一下代码的底层设计。



每个 Provider 独立熔断器: OpenAI 熔断不影响 Claude

极客时间

为什么每个 Provider 独立熔断器？OpenAI 挂了不代表 Claude 也挂了。共用的话一家失败会把其他正常的也拦住。

重试按异常类型区分，认证失败重试一百次也不会成功，只有“暂时性”的失败才值得重试。

3. 结构化日志与请求追踪

> 在 hify-common 中配置统一日志（com.hify.common.log）。logback-spring.xml 区分环境：开发环境控制台彩色输出，生产环境 JSON 格式输出到文件（按天滚动，保留 30 天）。创建 RequestLogInterceptor（HandlerInterceptor），记录每个请求的 method、path、status、耗时，慢请求（>1s）标 WARN。请求进入时生成 traceId 放入 MDC，请求结束时清理。日志格式：`%d{HH:mm:ss} [%thread] [%X{traceId}] %-5level %logger{20} - %msg%n`。

输出是：

复制代码

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <property name="LOG_DIR" value="${LOG_DIR:-logs}"/>
    <property name="APP_NAME" value="hify"/>
    <property name="LOG_PATTERN" value="%d{HH:mm:ss} [%thread] [%X{traceId}] %-5l

    <!-- ── 开发环境：控制台彩色输出 ──────────────────────────── -->
    <springProfile name="default,dev">
```

```
        <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
            <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
                <pattern>%clr(%d{HH:mm:ss}){faint} %clr([%thread]){magenta} %clr(
                <charset>UTF-8</charset>
            </encoder>
        </appender>

        <root level="INFO">
            <appender-ref ref="CONSOLE"/>
        </root>
    </springProfile>

    <!-- ── 生产环境：JSON 格式滚动文件输出 ──────────────────────── -->
    <springProfile name="prod">
        <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppen
            <file>${LOG_DIR}/${APP_NAME}.log</file>
            <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPol
                <fileNamePattern>${LOG_DIR}/${APP_NAME}.%d{yyyy-MM-dd}.log</fileN
                <maxHistory>30</maxHistory>
                <totalSizeCap>5GB</totalSizeCap>
            </rollingPolicy>
            <encoder class="ch.qos.logback.classic.encoder.PatternLayoutEncoder">
                <pattern>{"time":"%d{yyyy-MM-dd HH:mm:ss.SSS}","thread":"%thread"
                <charset>UTF-8</charset>
            </encoder>
        </appender>

        <appender name="FILE_ASYNC" class="ch.qos.logback.classic.AsyncAppender">
            <discardingThreshold>0</discardingThreshold>
            <queueSize>512</queueSize>
            <appender-ref ref="FILE"/>
        </appender>

        <root level="INFO">
            <appender-ref ref="FILE_ASYNC"/>
        </root>
    </springProfile>

</configuration>
```

traceId 的价值：一个对话请求从 Controller 进来，经过 Service、调 LLM、写数据库，可能产生十几条日志。一个 ID 串起整条链路，排查时不用大海捞针。

最终验收

三档全部搭完，回过头做一个整体检查：

- 启动项目，确认日志格式正确（有 traceId、有线程名）。
- 跑一遍 DemoItem 的 CRUD，确认请求日志过滤器在记录每个请求的 method、path、耗时。
- 检查日志里线程名是否有 `llm-` 和 `async-` 前缀的线程池就绪。

全部通过，Hify 后端基础组件全部就绪。

总结

这节课我们把 Hify 后端的全部基础组件搭完了。

- 第一档（跑起来）：建表 DDL、MapperScan 扫描路径、LLM / 异步线程池。
- 第二档（能写业务）：BaseEntity、分页封装、入参校验、时间序列化、Spring Cache、CRUD 标准流程。
- 第三档（健壮性）： HTTP 客户端（普通 + SSE）、熔断器（每个 Provider 独立）+ 重试、结构化日志 + 请求追踪。

最重要的方法论是咨询模式。在你还没完全想清楚的时候，先让 Claude Code 帮你梳理全景，你做判断和取舍，再进入执行。多了一个前置思考环节，但能帮你发现遗漏。后面每次进入新阶段都可以用。

这一讲内容很多，我想多说一句，**为什么要花这么大篇幅讲基础组件**。

这门课教的不只是 Claude Code 的使用。我还想让你知道，一个标准项目的基础组件应该长什么样，包括线程池隔离、熔断重试、Cache-Aside、traceId 串联、入参校验链路、统一时间序列化，这些东西不是课本上的概念，是每个生产级项目都要有的底层能力。很多工程师工作三五年，做的项目里这些东西是别人搭好的，自己从来没从零搭过，知道名字但说不清为什么要这么做。

这节课看完，你不只是用 Claude Code 搭好了 Hify 的基础组件，你还积累了一份可复用的基础组件清单，掌握了每个组件的设计理由。以后你开新项目，不管用不用 Claude Code，这套东西基本照搬就行。三档优先级、每个组件解决什么问题、参数为什么这么设，你都心里有数。这才是真正的积累，不是只会问 AI 问题，而是自己也有经验、有判断。

下一讲进入前端。程序员一般没有设计师的审美，没关系，让 Claude Code 来。我们会让它帮我们做 UI 设计、定色调、优化布局，把灰蒙蒙的空壳变成有品牌感的界面，同时把前端的基础组件封装好。

思考题

试一下咨询模式。打开 Claude Code，描述你正在做的一个项目，然后问它：“在开始写业务代码之前，我应该准备哪些基础组件？”看看它的建议里有没有你没想到的，并把它的建议和你的判断（哪些采纳、哪些不需要、为什么）分享在留言区。

期待你与我交流。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (5)

最新　精选



GaoMatrix

2026-04-24 来自山东

这个有视频就好了，VibeCoding视频演示是最好的，很多都是操作

作者回复: 后面有实操视频～

2



kyptonite

2026-06-11 来自重庆

本人是前端开发，看了这节课，基本老师每个发给AI的指令都不太懂，我就每一个都去问了AI是什么意思（让他把我当成Java初学者给我讲解），然后基本理解这些指令存在的意义了。
只能说学无止境，现在有了AI，方便很多。

> 作者回复: 是的，这就是最正确的学习姿势。哪个程序员不是这么过来的呢，我也是。
> 学习就是跟着主线走，不懂去问，懂了后回到主线，然后就都懂了。嘿嘿




1



神月

2026-04-30 来自江苏

我问AI： 帮我分析普通请求用 RestTemplate、流式 SSE 请求用 OkHttp EventSource 这个方案，有没有问题?
怎么得出如下答案跟老师不一样。
根据分析，你提出的方案存在以下致命问题：

方案问题
1. EventSource 的致命缺陷
EventSource 是浏览器端 JavaScript API，不是 Java 端的实现
不支持自定义请求头：LLM API 需要 Authorization: Bearer {token}，EventSource 无法设置
只支持 GET 请求：LLM API 需要 POST 传递大量 prompt 数据
2. RestTemplate 技术过时
已进入维护模式，Spring 官方推荐 WebClient
阻塞式 I/O，不适合高并发和流式场景
与 Spring Boot 3.x 生态不匹配
3. 混用增加复杂度
两套技术栈、两套配置、两套连接池
团队学习成本增加，维护困难

> 作者回复: 你用的是什么模型呢？ 不过不一样是正常的，同一个提示词，不同模型的答案是不一样的，同一个模型在不同的时间输出也是不一样的。这就是模型输出的随机性。
> 关键是我们能不能理解要做什么，对输出的内容有判断。

共 3 条评论>


1



s(t)

2026-06-29 来自上海

感觉这节课的提示词有点太专业了，是明确知道要做什么，细节怎么做后提出来的。要是我不太熟悉这些东西该怎么办？先咨询看看？

> 作者回复: 我自己确实是这样，我脑子里面知道做什么，怎么做了。然后去写提示词，做出来。
> 如果不熟悉的话，我觉得你可以跟ai 一起协同写出提示词。学会跟ai一起写出提示词，本身就是一个很不错的技能。我经常这么干。
> 项目中前端的提示词我就是这么干的。




星辞Sting

2026-05-02 来自浙江

挺好的，确实如克重所说 干了几年开发了，基础组件和能力的全貌是上面都不了解，没自己搭过

> 作者回复: 所以项目经验是很重要的，有完整的项目经验是很重要的。在项目经验中学效果更好。